Guía de entrevista
Senior React Native

Preguntas, respuestas y conceptos clave
Fabric, TurboModules, JSI, Hermes, estado, nativo, TypeScript, CI/CD y liderazgo

0. Glosario: los conceptos de la New Architecture

Antes de las preguntas, estos son los términos que más se repiten en entrevistas senior de React Native.

El Bridge (arquitectura antigua)

En la arquitectura original, JavaScript y el código nativo vivían en mundos separados y se comunicaban mediante el **Bridge**: un canal asíncrono que serializaba los mensajes a JSON, los enviaba por lotes y el otro lado los deserializaba. Problemas: es asíncrono (no puedes pedir un valor nativo y recibirlo al instante), la serialización cuesta CPU, y los mensajes se pueden acumular (cuello de botella en listas y gestos).

JSI (JavaScript Interface)

Es una capa escrita en C++ que permite a JavaScript tener **referencias directas a objetos nativos** (C++) y llamarlos **de forma síncrona**, sin serializar a JSON. Es la base de toda la New Architecture. Además es independiente del motor: funciona con Hermes, JavaScriptCore o V8. Gracias a JSI existen librerías como react-native-mmkv, Reanimated 3 o Vision Camera, que son muy rápidas porque evitan el bridge.

Fabric (nuevo sistema de renderizado)

Es el reemplazo del antiguo UIManager. Cambia cómo React Native convierte tu árbol de componentes en vistas nativas:

- El **Shadow Tree** (árbol de layout calculado con Yoga) ahora está en C++ y es **compartido** entre JS y nativo; es inmutable.
- Permite **renderizado síncrono** cuando hace falta (por ejemplo, medir una vista y reaccionar en el mismo frame).
- Soporta las funciones de **React concurrente**: Suspense, transiciones (`useTransition`) y priorización de actualizaciones.
- Mejor interoperabilidad y menos parpadeos de layout.

TurboModules (nuevos módulos nativos)

Son la evolución de los antiguos *Native Modules*. Diferencias clave:

- Se cargan de forma **perezosa (lazy)**: solo cuando se usan por primera vez, no todos al arrancar. Esto mejora el tiempo de inicio.
- Se comunican por **JSI**, así que las llamadas pueden ser síncronas y sin serialización JSON.
- Tienen una **interfaz tipada** (spec en TypeScript/Flow) que Codegen usa para generar código seguro.

Codegen

Herramienta que lee las especificaciones tipadas (specs) de tus TurboModules y componentes Fabric y **genera automáticamente el código C++/Obj-C/Java** de la interfaz. Así se detectan errores de tipos entre JS y nativo en tiempo de compilación, no en producción.

Hermes

Motor de JavaScript creado por Meta, optimizado para móviles. Compila el JS a **bytecode** en tiempo de build (AOT), por lo que la app arranca más rápido, usa menos memoria y pesa menos. Es el motor por defecto en React Native moderno.

Resumen comparativo

Aspecto	Arquitectura antigua	New Architecture
Comunicación JS-nativo	Bridge asíncrono con JSON	JSI: llamadas directas, pueden ser síncronas
Módulos nativos	Native Modules (todos cargados al inicio)	TurboModules (carga perezosa, tipados)
Renderizado	UIManager asíncrono	Fabric (Shadow Tree compartido en C++)
Tipado JS-nativo	Manual, errores en runtime	Codegen, errores en compilación
React concurrente	No soportado	Soportado

1. Arquitectura y rendimiento

P: ¿Cómo funciona la New Architecture (Fabric, TurboModules, JSI) y qué problemas del bridge resuelve?

La New Architecture elimina el bridge asíncrono basado en JSON. **JSI** da acceso directo y síncrono entre JS y C++; **Fabric** renderiza usando un Shadow Tree compartido en C++ y soporta React concurrente; **TurboModules** cargan los módulos nativos bajo demanda con interfaces tipadas generadas por **Codegen**.

- **Problemas que resuelve:** latencia por serialización JSON, imposibilidad de llamadas síncronas, arranque lento por cargar todos los módulos, cuellos de botella en gestos/animaciones y errores de tipos entre JS y nativo.
- **Qué decir en entrevista:** menciona también que la migración es gradual (hay una capa de interoperabilidad para librerías antiguas) y que debes verificar que tus dependencias soporten Fabric.

P: ¿Cómo diagnosticarías una lista lenta? (FlatList vs FlashList, getItemLayout, memoización, imágenes)

Primero **mido** antes de optimizar: React DevTools Profiler, Flipper/React Native DevTools y el monitor de rendimiento para ver FPS en JS y UI thread. Luego aplico mejoras según la causa:

- **FlashList (Shopify)**: recicla celdas como RecyclerView/UITableView; suele ser mucho más fluida que FlatList en listas grandes. Requiere `estimatedItemSize`.
- **FlatList bien configurada**: `keyExtractor` estable, `getItemLayout` si las alturas son fijas (evita medir), `windowSize`, `maxToRenderPerBatch`, `removeClippedSubviews`.
- **Memoización**: `React.memo` en el item, `useCallback` en `renderItem` y handlers; evitar funciones/objetos inline.
- **Imágenes**: tamaño adecuado, caché y placeholders (expo-image o react-native-fast-image), formato WebP, no cargar imágenes enormes en miniaturas.
- **Evitar trabajo pesado en render**: mover cálculos a `useMemo` o al servidor; no anidar listas verticales dentro de ScrollView.

```
<FlashList
  data={items}
  renderItem={renderItem}        // memoizado con useCallback
  estimatedItemSize={80}
  keyExtractor={(i) => i.id}
/>
```

P: ¿Cuándo usarías Hermes y qué cambia?

Casi siempre: es el motor por defecto. Hermes precompila el JavaScript a **bytecode** durante el build, lo que reduce el tiempo de arranque (TTI), el consumo de memoria y el tamaño de la app, a costa de no tener JIT. Considera alternativas solo si necesitas una API de JS que Hermes no soporte (cada vez menos) o librerías específicas.

- **Qué cambia:** el bundle se entrega como `.hbc`; el depurado se hace con Chrome DevTools via protocolo de Hermes; el profiling de JS se hace con su sampling profiler.
- **Detalle senior:** al no tener JIT, el código muy intensivo en cómputo puede ser más lento en picos, pero el arranque y la memoria compensan en la mayoría de apps.

P: ¿Cómo reduces el tiempo de arranque y el tamaño del bundle?

Ataco las tres fases del arranque: carga nativa, carga del bundle JS y primer render.

- **Hermes + bytecode** y **inline requires** (Metro) para cargar módulos solo cuando se usan.

- **TurboModules** lazy para no inicializar módulos nativos innecesarios.
- Reducir dependencias: analizar con `react-native-bundle-visualizer`, evitar librerías pesadas (moment.js por date-fns/dayjs, lodash por imports específicos).
- Diferir trabajo no crítico tras el primer render (`InteractionManager`, lazy screens con React Navigation).
- Mantener el **splash screen** hasta que los datos mínimos estén listos y evitar cadenas de peticiones en el arranque.
- Android: habilitar ProGuard/R8 y App Bundles; iOS: bitcode ya no aplica, pero sí revisar assets y arquitecturas.

2. React y estado

P: Diferencia entre useMemo, useCallback y React.memo, y cuándo NO usarlos.

Los tres evitan trabajo repetido, pero en distintos niveles:

- **useMemo(fn, deps)**: memoiza el *resultado* de un cálculo costoso.
- **useCallback(fn, deps)**: memoiza la *referencia de una función* para que no cambie entre renders.
- **React.memo(Componente)**: evita re-renderizar un componente si sus props no cambiaron (comparación superficial).
- **Cuándo NO usarlos:** en cálculos baratos (el costo de memoizar puede superar el de recalcular), cuando las dependencias cambian en cada render, o sin medir primero. `useCallback` solo sirve si el hijo está envuelto en `React.memo` o la función es dependencia de otro hook. Con React Compiler muchas memoizaciones manuales dejan de ser necesarias.

```
const Item = React.memo(({ item, onPress }) => (...));

const onPress = useCallback((id) => select(id), [select]);
const total = useMemo(() => calcularTotal(items), [items]);
```

P: ¿Cómo eliges entre Redux Toolkit, Zustand, Context o React Query?

Primero separo **tipos de estado** y elijo la herramienta para cada uno:

- **Estado del servidor** (datos de API): React Query / TanStack Query. Da caché, reintentos, invalidación, paginación y refetch en foco.
- **Estado global de cliente simple**: Zustand (poco boilerplate, selectores que evitan re-renders).
- **Estado global complejo con mucha lógica, middleware y depuración**: Redux Toolkit (devtools, convenciones claras para equipos grandes).
- **Context**: para valores que cambian poco (tema, idioma, sesión). No es ideal para estado que cambia con frecuencia porque re-renderiza a todos los consumidores.
- **Estado local**: `useState`/`useReducer` siempre que no necesite compartirse.

P: ¿Cómo evitas re-renders innecesarios?

Primero identifico con el Profiler cuáles son realmente problemáticos. Después:

- Colocar el estado lo más cerca posible de quien lo usa (*state colocation*).
- Selectores granulares (Zustand/Redux) para suscribirse solo a lo necesario.
- `React.memo`, referencias estables con `useCallback`/`useMemo` donde aporte.
- Dividir componentes grandes; pasar `children` en vez de recrear subárboles.
- No crear objetos/arrays/funciones nuevos en props de componentes memoizados.
- Usar claves (`key`) estables en listas.

P: Manejo de datos del servidor: caché, reintentos, offline-first.

Uso React Query para caché y reintentos automáticos (con backoff exponencial), y lo combino con persistencia local para soportar offline.

- **Caché**: `staleTime` y `gcTime` según qué tan frescos deben ser los datos.
- **Persistencia**: persistir la caché con MMKV o AsyncStorage (`persistQueryClient`), o SQLite/WatermelonDB/Realm para datos relacionales grandes.
- **Offline-first**: leer siempre de la base local, sincronizar en segundo plano, cola de mutaciones pendientes y estrategia de resolución de conflictos (last-write-wins o merge).
- **Conectividad**: NetInfo para detectar red y reanudar la cola al reconectar.
- **UX**: actualizaciones optimistas con rollback y estados claros de carga/error/vacío.

```
const { data } = useQuery({
  queryKey: ['feed', page],
  queryFn: () => api.getFeed(page),
  staleTime: 60_000,
  retry: 3,
});
```

3. Nativo e integración

P: ¿Cómo escribirías un módulo nativo (Swift/Kotlin)?

Con la New Architecture, creo un **TurboModule**:

- Defino la **spec tipada** en TypeScript (`NativeMiModulo.ts`).
- **Codegen** genera las interfaces nativas.
- Implemento el módulo en **Kotlin** (Android) y en **Swift/Obj-C++** (iOS) siguiendo la interfaz generada.
- Lo registro y lo consumo desde JS con `TurboModuleRegistry`.
- Con Expo puedo usar el **Expo Modules API**, que simplifica mucho escribir módulos en Swift/Kotlin.

```
// NativeDeviceInfo.ts
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  getBatteryLevel(): number;               // síncrono (JSI)
  getDeviceName(): Promise<string>;        // asíncrono
}
export default TurboModuleRegistry.getEnforcing<Spec>('DeviceInfo');
```

P: Expo managed vs bare workflow: trade-offs y config plugins.

Hoy la distinción es menos rígida: con **Continuous Native Generation (prebuild)**, Expo genera las carpetas `ios/` y `android/` a partir de tu configuración.

- **Managed**: menos mantenimiento, actualizaciones de SDK más simples, EAS Build/Update. Limitación histórica: código nativo propio (hoy se resuelve con config plugins y dev clients).
- **Bare**: control total del proyecto nativo, pero tú mantienes Gradle, Xcode, Podfile y las actualizaciones de React Native.
- **Config plugins**: funciones que modifican la configuración nativa (Info.plist, AndroidManifest, Gradle, Podfile) durante el prebuild, de forma reproducible y sin editar archivos nativos a mano.

P: Deep linking, notificaciones push, permisos y background tasks.

Cada uno tiene particularidades por plataforma:

- **Deep linking**: esquemas personalizados, *Universal Links* (iOS) y *App Links* (Android) con archivos de verificación de dominio. React Navigation permite mapear URLs a pantallas con `linking`.

- **Push**: APNs (iOS) y FCM (Android), vía Firebase o Expo Notifications. Manejar tres estados: app en primer plano, segundo plano y cerrada. Pedir permiso en un momento oportuno (iOS exige consentimiento; Android 13+ también).
- **Permisos**: solicitar en contexto, explicar el porqué, manejar el estado 'denegado permanentemente' llevando al usuario a Ajustes.
- **Background tasks**: muy limitados por el sistema (iOS BGTaskScheduler, Android WorkManager/Headless JS). Usar expo-task-manager o react-native-background-fetch y no depender de ejecución garantizada.

P: Diferencias de comportamiento entre iOS y Android que te hayan costado.

Buena pregunta para mostrar experiencia real. Ejemplos habituales:

- **Teclado**: `KeyboardAvoidingView` se comporta distinto (behavior padding vs height); Android usa `windowSoftInputMode`.
- **Sombras**: iOS usa `shadow*`; Android usa `elevation`.
- **Botón atrás** de Android y gestos de navegación en iOS.
- **Permisos y notificaciones**: canales de notificación en Android; flujos de consentimiento distintos.
- **Fuentes, status bar, safe areas, notch y edge-to-edge** (Android 15).
- **Rendimiento y memoria**: dispositivos Android de gama baja exigen más cuidado con imágenes y listas.
- Consejo: nombra un caso concreto, cómo lo detectaste y cómo lo solucionaste (`Platform.select`, archivos `.ios.tsx`/`.android.tsx`).

4. TypeScript

P: Tipado de navegación (React Navigation), props genéricas y unknown vs any.

Navegación: defino un tipo con los parámetros de cada ruta y lo uso en `createNativeStackNavigator` y en los hooks, para que `navigate` sea seguro.

```
type RootStackParamList = {
  Home: undefined;
  Profile: { userId: string };
};

declare global {
  namespace ReactNavigation {
    interface RootParamList extends RootStackParamList {}
  }
}

navigation.navigate('Profile', { userId: '42' }); // tipado
```

Props genéricas: componentes reutilizables que conservan el tipo del dato.

```
type ListProps<T> = {
  data: T[];
  renderItem: (item: T) => React.ReactElement;
};
function List<T>({ data, renderItem }: ListProps<T>) {
  return <>{data.map(renderItem)}</>;
}
```

unknown vs any: `any` desactiva la comprobación de tipos y contagia. `unknown` acepta cualquier valor pero te obliga a *estrechar* el tipo (con `typeof`, `instanceof` o validación) antes de usarlo. Regla: usa `unknown` para datos externos; evita `any`.

P: Cómo tipas respuestas de API (Zod, validación en runtime).

TypeScript solo existe en compilación: una respuesta de API puede no coincidir con lo que declaraste. Por eso valido en **runtime** con **Zod** y *deduzco* el tipo del esquema, manteniendo una única fuente de verdad.

- Beneficios: detecta cambios inesperados del backend, evita crashes por `undefined` y permite transformar datos (fechas, defaults).
- Alternativas: Valibot, io-ts, o generar cliente tipado desde OpenAPI/GraphQL (codegen).

```
import { z } from 'zod';

const UserSchema = z.object({
  id: z.string(),
  name: z.string(),
  email: z.string().email(),
});
type User = z.infer<typeof UserSchema>;

const user = UserSchema.parse(await res.json()); // lanza si no coincide
```

5. Calidad y entrega

P: Estrategia de testing: Jest, React Native Testing Library, Detox/Maestro.

Sigo la pirámide de pruebas:

- **Unitarias (Jest)**: lógica pura, reducers, utilidades, hooks.
- **Componentes (React Native Testing Library)**: pruebo comportamiento visible para el usuario (`getByText`, `fireEvent`), no detalles de implementación. Mock de red con MSW.
- **E2E (Detox o Maestro)**: flujos críticos (login, compra). Detox es gray-box y rápido pero más complejo de configurar; Maestro usa YAML, es muy sencillo y multiplataforma.
- Cobertura útil, no obsesiva; ejecutar en CI en cada PR.

P: CI/CD: EAS o Fastlane, OTA updates y versionado.

El pipeline ideal: lint + typecheck + tests en cada PR, build automático y distribución a TestFlight/Play Internal.

- **EAS Build/Submit**: builds en la nube sin mantener máquinas macOS; **Fastlane**: más control y personalización, típico en proyectos bare.
- **OTA updates** (EAS Update, CodePush alternativas): permiten publicar cambios de JS/assets sin pasar por la tienda; no sirven para cambios nativos. Usar canales (staging/production), rollouts graduales y *runtimeVersion* para evitar incompatibilidades.
- **Versionado**: semver para la versión visible y *build number* autoincremental; ramas y tags de release; changelog automático.

P: Monitoreo: Sentry, Crashlytics, performance en producción.

Sin observabilidad no sabes qué falla. Combino:

- **Crashes y errores**: Sentry o Crashlytics, con *source maps* y símbolos subidos en CI para ver stack traces legibles.
- **Performance**: tiempo de arranque, tiempo de pantalla, frames lentos/congelados (Sentry Performance, Firebase Performance).
- **Analítica y breadcrumbs** para reproducir errores; alertas por regresiones tras cada release.
- **Feature flags** para apagar funciones problemáticas sin republicar.

P: Seguridad: almacenamiento de tokens (Keychain/Keystore), certificate pinning.

Nunca guardo tokens en AsyncStorage (no está cifrado).

- **Almacenamiento seguro**: `react-native-keychain` o `expo-secure-store` (iOS Keychain / Android Keystore).
- **Tokens**: access token de vida corta + refresh token; rotación y revocación; biometría para operaciones sensibles.
- **Certificate pinning**: protege contra ataques man-in-the-middle fijando el certificado o clave pública del servidor (react-native-ssl-pinning, TrustKit); planifica la rotación de certificados.
- **Otros**: ofuscación (R8/Hermes bytecode), detección de root/jailbreak si aplica, no incluir secretos en el bundle (todo lo del cliente es extraíble), HTTPS estricto.

6. Liderazgo (lo que distingue a un senior)

Estas preguntas no tienen una única respuesta correcta; evalúan criterio. Usa el método **STAR** (Situación, Tarea, Acción, Resultado) y cuantifica el impacto.

P: Cuéntame una decisión técnica difícil y cómo la defendiste.

Estructura sugerida:

- **Contexto**: qué problema y qué restricciones (tiempo, equipo, negocio).
- **Opciones**: al menos dos alternativas con sus trade-offs.
- **Decisión y argumentos**: datos, prototipos o benchmarks que respaldaron la elección; cómo alineaste a stakeholders.
- **Resultado y aprendizaje**: métricas, y qué cambiarías hoy. Reconocer errores suma puntos.
- *Ejemplo:* migrar de Redux a React Query + Zustand, o adoptar la New Architecture gradualmente.

P: ¿Cómo haces code review y mentoría a juniors?

Mi objetivo es mejorar el código *y* a la persona.

- **Code review**: revisar primero el diseño y la corrección, luego estilo (automatizado con lint/prettier). Comentarios claros, con el porqué y alternativas; distinguir bloqueantes de sugerencias; reconocer lo bien hecho.
- **Mentoría**: pair programming, explicar el razonamiento en vez de dar la solución, tareas con dificultad progresiva, feedback frecuente y documentación de decisiones (ADRs).
- **Cultura**: PRs pequeños, definición de 'hecho' compartida, seguridad psicológica para preguntar.

P: ¿Cómo estructurarías un monorepo o un design system compartido?

Un monorepo con herramientas como **Nx** o **Turborepo** y **pnpm/yarn workspaces**:

- **Design system**: *design tokens* (colores, espaciado, tipografía), componentes primitivos accesibles, documentados en Storybook y con tests visuales; versionado y changelog.
- **Cuidados con Metro**: configurar `watchFolders` y resolución de módulos; evitar duplicar versiones de React/React Native.
- **Beneficios**: reutilización, cambios atómicos, caché de builds en CI. **Costos**: complejidad de herramientas y tiempos de CI si no hay caché.

```
/apps
  /mobile        (React Native / Expo)
  /web           (opcional)
/packages
  /ui            (design system: componentes + tokens)
  /api-client    (cliente tipado, esquemas Zod)
  /utils
  /config        (eslint, tsconfig, prettier)
```

P: Un bug crítico en producción: ¿cómo lo manejas?

Priorizo **mitigar primero, entender después**:

- **1. Detectar y comunicar**: confirmar alcance (usuarios, versiones, plataformas) con Sentry/Crashlytics; avisar a stakeholders y soporte.
- **2. Mitigar**: feature flag, rollback, OTA con el arreglo o hotfix si es nativo (con revisión acelerada de tiendas).
- **3. Corregir**: reproducir, arreglar con test de regresión, revisión rápida y despliegue gradual.
- **4. Postmortem sin culpas**: causa raíz, línea de tiempo y acciones preventivas (tests, alertas, procesos).

Ejercicio práctico: feed con scroll infinito, caché offline y manejo de errores

Esta es una forma de plantear la solución explicando tus decisiones:

- **Datos**: `useInfiniteQuery` (React Query) con paginación por cursor; caché persistida con MMKV.
- **Lista**: FlashList con `onEndReached`, `onEndReachedThreshold` y pull-to-refresh; items memoizados.
- **Offline**: mostrar datos en caché inmediatamente, banner de 'sin conexión' con NetInfo y reintento automático al reconectar.
- **Errores**: estados diferenciados (carga inicial, error con botón de reintentar, error al paginar en el footer, lista vacía).
- **Validación**: esquemas Zod para las respuestas.
- **Calidad**: tests con RNTL + MSW, y métricas de rendimiento (FPS, tiempo a primer contenido).

```
const { data, fetchNextPage, hasNextPage, isFetchingNextPage, refetch, isError } =
  useInfiniteQuery({
    queryKey: ['feed'],
    queryFn: ({ pageParam }) => api.getFeed(pageParam),
    initialPageParam: null as string | null,
    getNextPageParam: (last) => last.nextCursor ?? undefined,
  });

const items = useMemo(() => data?.pages.flatMap((p) => p.items) ?? [], [data]);

<FlashList
  data={items}
  renderItem={renderItem}
  estimatedItemSize={120}
  onEndReached={() => hasNextPage && fetchNextPage()}
  onEndReachedThreshold={0.5}
  ListFooterComponent={isFetchingNextPage ? <Spinner /> : null}
/>
```

Consejos finales para la entrevista

- Siempre explica el **porqué** y los **trade-offs**, no solo el qué.
- Mide antes de optimizar; menciona herramientas concretas.
- Si no sabes algo, di cómo lo investigarías.
- Prepara 2 o 3 historias reales (STAR) de proyectos, errores y liderazgo.